

Page 02 — Articles /articles.html

Audited: 6 September 2026 · live site + repository `bccody03/blog-site @ main 88fdcd1` (merged into the audit branch)
Live URL: `https://blakecody.com/articles.html` — HTTP 200, Last-Modified: 2026-09-04 , repo copy verified byte-identical to live
Method: static HTML parse · headless Chromium render with a 23-post feed injected · pagination behaviour tested by clicking through · live HTTP + redirect probes · full read of the updated `app.js`
Status: Audit only — **no code changes applied**. Every entry is a proposed change with the exact edit.

16 CHANGES PROPOSED	1 CRITICAL	6 HIGH	9 MEDIUM / LOW	17 PAGE-01 FIXES VERIFIED
------------------------	---------------	-----------	-------------------	---------------------------------

First: 17 of the page-01 findings are done, and done correctly VERIFIED

Since the homepage audit, `main` has moved (commit `88fcd1`, "SEO audit pass 1"). I re-verified each fix against the live site rather than the diff. All of these carry over to this page:

ID	Fix	Verification
T1	Enforce HTTPS	✓ <code>curl -I http://.../articles.html</code> → 301 → <code>https://</code>
T3	Fake sample posts removed	✓ <code>SAMPLE_POSTS</code> now gated behind <code>?demo</code> ; the error path sets an honest "Latest posts are on Substack →" state and fires a <code>feed-error</code> event
T4	Fonts non-blocking	✓ <code>rel="preload" as="style"</code> + <code>onload</code> swap, with a <code><noscript></code> fallback
T8	Internal links to <code>/</code>	✓ brand, Home and the "← Home" eyebrow all point at <code>/</code>
T9	Sitemap	✓ <code>lastmod</code> now 2026-09-04 and accurate; <code><priority></code> dropped; <code>privacy.html</code> added
T10	Image weight	✓ logo 45 KB → 4 KB at 60×60 with <code>width / height</code> · og-image PNG → JPEG, 91 → 46 KB · cover 443 → 207 KB · hero 92 → 79 KB
T11	Scripts deferred	✓ <code><script src="app.js?v=24" defer></code> in the head
T12	Lazy subscribe iframe	✓ <code>loading="lazy"</code>
T14	<code>rel="me"</code>	✓ on the Substack nav link
C5	Post titles → <code>h3</code>	✓ confirmed in the rendered DOM
C7	OG completeness	✓ <code>og:locale</code> , <code>og:image:alt</code> added
C8	Thumbnail alt text	✓ rendered alt = the post title, plus <code>width / height</code>
K1	GoatCounter events	✓ <code>track()</code> helper + delegated outbound listener + <code>feed-error</code>
K2	HTTPS analytics URL	✓ <code>https://gc.zgo.at/count.js</code>
K3	UTM tagging	✓ rendered card href carries <code>utm_campaign=articles&utm_content=card</code>
K6	Form source attribution	✓ <code>entryContext()</code> captures landing page, referrer and query into <code>sessionStorage</code>
K9	Privacy page	✓ <code>/privacy.html</code> live, linked in the footer, in the sitemap

That is a genuinely strong implementation pass. This report only covers what is still open **on this page**.

Do these in this order

P0 — this week

1. **T16** Give pagination real URLs. Right now posts 10 and older have no address at all.
2. **C13 C15** Rewrite the title and the H1 — both currently spend themselves on the word "Articles".
3. **T18** Add `CollectionPage` + `ItemList` + `BreadcrumbList` schema. The homepage has a five-node graph; this page has none.

P1 — this month

4. **C14 C16 C19** Meta description, heading order, and 60–100 words of intro copy
5. **K10 K11** Track pagination depth and card position — you cannot currently tell if anyone ever reaches page 2
6. **C18** Add topic grouping; you already have the taxonomy on the Reflect page
7. **T19 T17** RSS link on the archive; confirm how deep the feed actually reaches

P2 — the one that dissolves four of the above

8. **C4** Host the essays on blakecody.com. **T16, T2-A, C17 and half of C18 all disappear the moment articles have their own URLs on your domain.** Do not build elaborate client-side pagination URL handling if you are going to do this — it would be work you throw away.

A. Technical

T16 Pagination creates no URLs — posts 10 and older have no address

CRITICAL

NOW

`articles.html` sets `data-page-size="9"`, and `renderPagination()` builds the controls as **buttons, not links**:

```
html += `<button class="page-btn page-num..." data-page="${i}"...>${i}</button>`;
```

I rendered the page with a 23-post feed and clicked through to page 2. Measured:

```
pager elements:  BUTTON ←  BUTTON 1  BUTTON 2  BUTTON 3  BUTTON →
                  href: null on every one

after clicking page 2:
  location.href    unchanged    (still /articles.html)
  location.search  ""
  location.hash    ""
  history.length   2             (unchanged — no history entry pushed)
  content          posts 10–18 now displayed
```

WHY IT MATTERS

Four separate consequences, all from the same cause:

1. **Crawlers never see past post 9.** Googlebot renders JavaScript but it does not click buttons. There is no URL for page 2, so there is no way for any crawler to reach posts 10 and older. On a page titled "Every essay and dispatch", most of the essays are invisible.
2. **Nothing beyond page 1 can be linked or shared.** No reader can send someone "the page with that essay from March".
3. **The back button is broken.** No history entry is pushed, so pressing Back from page 3 leaves the archive entirely instead of returning to page 2.
4. **No `rel="next"` / `rel="prev"`**, because there is nothing to point them at.

FIX — THE INTERIM VERSION

Make the pager real links with a query parameter, and read it on load:

```
// renderPagination — links, not buttons
html += `<a class="page-btn page-num${i === cur ? " active" : ""}"
  href="?page=${i}" data-page="${i}">${i}</a>`;

// on click, intercept and push state instead of reloading
history.pushState({ page: p }, "", p === 1 ? "articles.html" : "?page=" + p);

// on load, honour the parameter
const start = parseInt(new URLSearchParams(location.search).get("page"), 10) || 1;
renderPage(start);

// and handle Back
addEventListener("popstate", (e) => renderPage((e.state && e.state.page) || 1));
```

Then add `<link rel="canonical" href="https://blakecody.com/articles.html?page=N">` per page — or, simpler and better for a small archive, keep the canonical on page 1 and add `rel="next"` / `rel="prev"`.

FIX — THE VERSION THAT ACTUALLY SOLVES IT

For an archive of this size, **paginating at all is the wrong call**. Nine per page across 20–30 essays gives you three pages, none of which a crawler can reach, in exchange for saving a reader a scroll. Either:

- **Show all posts on one page** — set `data-page-size="0"`, delete the pager. One URL, every essay crawlable, no history problem, no canonical question. This is a one-character change and it fixes the whole finding.
- **Or do C4** and give each essay its own URL, at which point the archive is a real index and pagination is a genuine paging concern rather than a content-hiding one.

Given that you are likely to do **C4** eventually, I would set `data-page-size="0"` today and not build the `pushState` machinery at all.

T2-A The archive serves 50 words and an empty div HIGH

NOW

Measured static text in `<body>`: **50 words** — navigation, an H1, one sentence, "Loading...", the subscribe block and the footer. Rendered after JS: 296 words, 9 cards.

```
<div class="state" id="state">Loading...</div>
<div class="post-list" id="post-list" data-page-size="9"></div>
```

WHY IT MATTERS

This is the same root cause as **T2** on the homepage, but the consequence is worse here. The homepage has an Explore grid, an app block and a subscribe section that exist in HTML regardless. This page's *entire reason to exist* is the list, and the list is not in the HTML. To a crawler that does not run JavaScript — Bing, and effectively every AI crawler — `articles.html` is a 50-word page that says "Loading...".

FIX

Same build-time prerender proposed in **T2**: a scheduled GitHub Action fetches the Substack RSS and writes the cards into the HTML. Interim, add a `<noscript>` list of the most recent titles and links so there is something real in the markup.

T18 No structured data on this page HIGH

NOW

Measured across the site:

Page	JSON-LD blocks
index.html	1 — a five-node graph: Person, WebSite, Book, SoftwareApplication, WebPage ✓
articles.html	0
book.html, reflect.html, about.html, privacy.html, 404.html	0 (covered in their own reports)

The homepage schema was implemented well. It just did not travel to the other pages.

FIX

An archive wants three things: what kind of page it is, what it lists, and where it sits. Add before `</head>`:

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@graph": [
    { "@type": "CollectionPage",
      "@id": "https://blakecody.com/articles.html#webpage",
      "url": "https://blakecody.com/articles.html",
      "name": "Essays by Blake Cody",
      "description": "Every essay and dispatch on discipline, love and living with intention.",
      "isPartOf": { "@id": "https://blakecody.com/#website" },
      "about": { "@id": "https://blakecody.com/#person" },
      "inLanguage": "en-US" },
    { "@type": "BreadcrumbList",
      "itemListElement": [
        { "@type": "ListItem", "position": 1, "name": "Home",
          "item": "https://blakecody.com/" },
        { "@type": "ListItem", "position": 2, "name": "Articles" }
      ] }
  ]
}
```

Note the `@id` references reuse the nodes the homepage already defines — that is what makes it one graph rather than five disconnected ones.

Then add an `ItemList` of the posts, generated alongside the cards in **T2-A**'s prerender step. Generating it client-side is possible but pointless: if JavaScript has to run to produce your schema, the crawlers that most need the schema will not see it.

The `BreadcrumbList` also gives you the breadcrumb display in search results, and this page already has the visual breadcrumb to match it — `<p class="eyebrow"><a href="/">← Home</a></p>`.

T17 How deep does the feed actually reach? MEDIUM

`loadFromSubstack()` requests `count=50` from `rss2json`, which is generous. But the ceiling is not `rss2json` — it is Substack's own RSS feed, which typically serves only a recent window (commonly the last ~20 posts) regardless of what you ask for.

If that is the case here, then "Every essay and dispatch, newest first" is not true: essays older than the feed window are silently absent, with nothing on the page to say so.

VERIFY

I could not test this — `api.rss2json.com` is blocked by this environment's egress policy. Count the cards on the live archive and compare against your Substack archive page. If they differ, you have a content gap.

FIX IF CONFIRMED

The prerender pipeline (**T2-A**) solves it permanently: fetch on a schedule and **accumulate** into a local JSON file rather than replacing it, so posts that fall out of the feed window stay in your archive. This is another reason to move the fetch to build time.

T19 No RSS link on the page where it belongs most LOW

`index.html` now carries `<link rel="alternate" type="application/rss+xml">` ✓. `articles.html` does not — and the archive is exactly where a reader or a feed reader looks for it.

```
<link rel="alternate" type="application/rss+xml"
      title="Blake Cody on Substack" href="https://bccody.substack.com/feed" />
```

Add it to every page, or at minimum to this one. Consider a visible "Subscribe by RSS" link next to the pager too — the audience for a personal essay archive skews toward people who still use feed readers.

T20 `og-image.png` now returns 404 LOW

Verified live: `og-image.jpg` → 200 (46 KB) ✓, `og-image.png` → 404.

The meta tags were all updated correctly, so new shares are fine. But any previously shared link whose preview was cached against the old PNG, and any external page referencing it, now points at a missing file.

FIX

GitHub Pages cannot issue a redirect for a static file, so the fix is simply to leave a copy: keep `og-image.png` in the repo alongside the JPEG. It costs 46 KB of repository space and nothing at runtime, since no page references it any more.

T21 `CONFIG.maxPosts` is now dead config LOW

`maxPosts: 12` is read only by `postLimit()`, which applies when a list has no `data-limit`. The homepage sets `data-limit="3"`; this page uses `data-page-size="9"` and paginates the full array. So `maxPosts` currently affects nothing.

Harmless today, misleading later — someone editing this file will reasonably assume changing it does something. Either delete it, or wire it to `count=` in `loadFromSubstack()` where a post cap actually belongs.

B. Content & text

C13 The title is 21 characters and says nothing HIGH

NOW

```
<title>Articles – Blake Cody</title> 21 characters of roughly 60
```

"Articles" is a navigation label, not a search term. Nobody types it, and it tells Google nothing about what these articles are about.

CHANGE TO

```
All Essays by Blake Cody – Discipline, Love & Intentional Living 63 chars
```

Or, if you would rather lead with the topic than the name:

```
Essays on Discipline, Love & Living with Intention | Blake Cody 62 chars
```

Either version gives this page a chance at non-brand queries, which the homepage cannot compete for and should not try to (see C11).

C15 The H1 is one word HIGH

NOW

```
<h1 class="archive-title">Articles</h1>
<p class="archive-sub">Every essay and dispatch, newest first.</p>
```

The homepage H1 was fixed well — it now reads "Lead with Love, Live with Intention" instead of just the name. This page has the same problem the homepage had, and the same fix applies.

CHANGE TO

```
<h1 class="archive-title">Essays on discipline, love and living with intention</h1>
<p class="archive-sub">Everything I've written, newest first – free to read.</p>
```

If that is too long for the design at this type size, "Every essay, newest first" is still a large improvement over "Articles": it is a phrase rather than a label.

C14 The meta description uses 87 of ~155 characters MEDIUM

NOW

```
Every essay and dispatch from Blake Cody on discipline, love and living
with intention. 87 chars
```

Better than it was — the topics are in there now. But it stops before giving anyone a reason to click, and it wastes 68 characters of pixel budget.

CHANGE TO

```
Every essay Blake Cody has written on discipline, love and living with
intention – free to read, newest first. New writing most weeks. 137 chars
```

"Free to read" and "new writing most weeks" are the two facts that make an archive worth clicking rather than bookmarking-for-later. If you know the count, use it: "All 24 essays..." is stronger than "every essay", because a number is a promise of substance.

C16 The heading order skips a level, then goes backwards MEDIUM

NOW — MEASURED IN THE RENDERED DOM

```
H1 Articles
H3 Essay number 1 ...      <- H1 jumps straight to H3, no H2
H3 Essay number 2 ...
... (nine of them)
H2 Stay up to date in your inbox  <- an H2 arriving after nine H3s
```

The **C5** fix (post titles from `h2` to `h3`) was right and fixed the homepage properly, where a "Latest articles" H2 sits above the list. This page has no such H2, so lowering the titles to H3 left a gap.

CHANGE TO

Give the list the H2 it is missing. The subtitle is already there and already says the right thing — just promote it:

```
<h1 class="archive-title">Essays on discipline, love and living with intention</h1>
<h2 class="archive-sub">Everything I've written, newest first</h2>
<h3> ... each post ... </h3>
```

Style `.archive-sub` however you like — the visual weight does not have to follow the tag. That gives a clean H1 → H2 → H3 tree, with the subscribe H2 as a sibling of the list H2, which is correct.

C18 No topic grouping — and you already have the taxonomy MEDIUM

NOW

The archive is a single reverse-chronological list. There are no categories, no tags, no topic pages. For search, that means there is no page on the site targeting any specific subject — the archive is one URL competing for everything at once, which in practice means it competes for nothing.

THE THING YOU ALREADY BUILT

`reflect.html` has topic chips — `.cat` elements with `data-cat` attributes — that let a reader pick a subject before writing to you. That taxonomy already reflects how you think about your own subject matter. It is the obvious grouping for the essays too.

CHANGE TO

- **Short term:** add filter chips to the archive using the same categories, matched against post titles/tags from the feed. Client-side filtering, no new URLs — useful for readers, neutral for SEO.
- **Real version (needs C4):** once essays live on your domain, each topic becomes a page — `/topics/discipline`, `/topics/intention` — with an intro paragraph and the essays beneath it. **These are the pages that would actually rank for non-brand queries.** The homepage never will; the archive alone probably will not; topic hubs are the standard answer and they work.

C19 One sentence of body copy on the whole page MEDIUM

"Every essay and dispatch, newest first." is the entire prose content of the archive. Nothing frames what the essays cover, who they are for, or why someone should start reading.

CHANGE TO

60–100 words between the H1 and the list. Not keyword filler — a real short introduction: what you write about, roughly how often, what a new reader should start with. Link two or three specific essays by name inside that paragraph. That gives you internal links with descriptive anchor text, which the card grid currently does not provide, and it gives Google a passage that describes the collection.

C17 A page called "Articles" that contains no articles HIGH → site-wide C4

Every card still links to `bccody.substack.com` with `target="_blank"`. Verified on a rendered card:

```
https://bccody.substack.com/p/post-1
?utm_source=blakecody.com&utm_medium=referral
&utm_campaign=articles&utm_content=card
```

The UTM tagging (K3) is correctly implemented and means you can now *measure* the traffic you export. It does not change that you are exporting it.

This is the same finding as C4 on the homepage, and I am repeating it here only because this is the page where it is most visible: the archive is the single strongest argument for hosting the essays on your own domain. It already has the layout, the pagination and the design. It is a fully built index pointing at someone else's website.

See the P2 note above: doing this dissolves T16, T2-A, half of C18 and this finding, all at once.

C. Tracking & measurement

K10 Pagination clicks are not tracked — you cannot tell if page 2 is ever reached MEDIUM

NOW

`renderPage()` and the pagination click handler contain no `track()` call. The K1 implementation covers outbound clicks, chapter requests, reflections and feed errors — all correct — but not this.

WHY IT MATTERS

This is the number that decides T16. If nobody ever clicks past page 1, pagination is pure cost — it hides content from crawlers and buys nothing — and you should set `data-page-size="0"` and be done. If people do page through, that tells you the archive is being browsed rather than bounced from, which is worth knowing on its own.

FIX

```
// inside the pagination click handler, after parsing p
track("archive-page-" + p, "Archive page " + p);
```

One line. Give it two weeks and the answer to T16 is a fact rather than a judgement call.

K11 Card clicks do not record which card MEDIUM

NOW

```
const where = a.id ? a.id : a.closest(".post") ? "card" : "other";
track("click-substack-" + where, "Substack click");
```

Every one of the nine cards on every page fires the identical `click-substack-card` event. You can see that the archive sends traffic to Substack; you cannot see whether readers click the newest post, scan to something further down, or hunt for a specific title.

FIX

```
const card = a.closest(".post");
if (card) {
  const idx = [...card.parentElement.children].indexOf(card) + 1;
  track("click-substack-card-" + idx, "Substack click, position " + idx);
}
```

If clicks cluster on position 1, the archive is functioning as "show me the latest" and the homepage's three-post list is doing the same job — consider whether you need both. If they spread down the list, readers are genuinely browsing, which argues for C18's topic grouping and against pagination.

K12 Search Console follow-ups specific to this page LOW

- Run **URL Inspection** → **Test live URL** → **View rendered HTML** on `/articles.html` and count the post cards. That confirms whether Google's renderer sees 9 cards, and it will not see more — which is **T16** demonstrated rather than argued.
- Filter the Performance report to `/articles.html` and look at the query list. If it is entirely brand terms, that is **C13** and **C18** quantified.
- Re-submit the sitemap now that `lastmod` is accurate — the values changed on 4 September and Google will not have re-read it on its own yet.

K13 Verified working on this page — leave alone PASS

- Outbound Substack clicks fire `click-substack-card` ✓
- UTM tagging is correct and page-specific (`utm_campaign=articles`) ✓
- `feed-error` fires on a failed feed and the visible state is honest ✓
- Entry context (landing page, referrer, query) is captured to `sessionStorage` ✓
- GoatCounter loads over HTTPS ✓

Evidence log

Check	Result
Repo vs live	Byte-identical after merging <code>origin/main 88fdcd1</code> — this audit reflects what is deployed
<code>curl -I http://.../articles.html</code>	301 → <code>https://blakecody.com/articles.html</code> — T1 confirmed fixed
Static text in <code><body></code>	50 words · rendered after JS: 296 words, 9 cards
Title / description	21 chars / 87 chars
Headings (rendered)	H1 → H3 ×9 → H2 — level skipped, then reversed
Pagination (23-post feed)	5 <code><button></code> elements, <code>href: null</code> on all · after clicking page 2: URL, search, hash all unchanged; <code>history.length</code> unchanged at 2
Card href (rendered)	<code>.../p/post-1?</code> <code>utm_source=blakecody.com&utm_medium=referral&utm_campaign=articles&utm_content=card</code>
Thumbnail alt (rendered)	= the post title, with <code>width="480" height="320"</code> — C8 confirmed fixed
JSON-LD	<code>index.html</code> : 1 block, 5 nodes · <code>articles.html</code> : 0
Feed request	<code>count=50</code> to <code>rss2json</code> ; real ceiling is Substack's own feed window
Assets	logo.png 4 KB @ 60×60 · og-image.jpg 46 KB (200) · og-image.png 404 · cover.jpg 207 KB · hero-bg.jpg 79 KB
Not verifiable here	<code>api.rss2json.com</code> blocked by egress policy — T17 feed depth needs manual confirmation

Page 02 of 6 complete. Remaining: `book.html` · `reflect.html` · `about.html` · `404.html` · plus the new `privacy.html`, which did not exist when this audit was scoped.

Site-wide findings raised on page 01 and still open — **T2** (client-side rendering), **C4** (articles hosted off-domain), **T5** (schema on pages other than the homepage), **T13** (10-minute cache ceiling) — are cross-referenced rather than repeated. Everything else from page 01 is verified fixed; see the table at the top.

Companion document: `ui/changelog-02-articles.pdf` — visual design findings for the same page (pending).

